NEF 3002- Applied Project 2

Winter Block Group 1

FINAL REPORT

SMART LOCKER SYSTEM

Team Members:

Bryan France s3892805

Peter Hans s4643570

Supervisor: Dr.Assefa K Teshome

Sponsor: A/Prof Thinh Nguyen

1 Introduction 3
1.1 Scope Review 3
2 System Requirements Review 4
2.1 Function Requirements Review 4
2.2 Non-Functional Requirement Review 5
3 Database 6
3.1 Database Review 6
3.2 Database layout and Functionality 6
4 System Architecture 8
4.1 User App System 8
4.1.1 Technologies Used 8
4.1.2 UI (User Interface) Descriptions 9
4.1.3 App Subsystems Explained 13
4.2 Locker Bank System 28
4.2.1 Technologies Used 28
4.2.2 UI (User Interface) Descriptions 29
4.2.3 Locker Bank Subsystems Explained 30
5 Challenges and Problems 35
6 Student Contributions 37
7 References 41

Image 3.2.1 Firestore document orientated database diagram.
Image 4.1.2.1 Splash Screen animation fade in fade out.
Image 4.1.2.2 Welcome Screen animation.
Image 4.1.2.3 Welcome Screen.
Image 4.1.2.4 Login Page Screen.
Image 4.1.2.5 Sign Up Screen.
Image 4.1.2.6 Dashboard Screen.
Image 4.1.2.7 Profile Screen.
Image 4.1.2.8 Edit Profile Screen.
Image 4.1.3.1 signup_screen.dart imports.
Image 4.1.3.2 signup_screen.dart class.
Image 4.1.3.3 signup_screen.dart body.
Image 4.1.3.4 form_header_widget.dart.
Image 4.1.3.5 signup_form_widget.dart.
Image 4.1.3.6 signup_form_widget.dart Text Form Field widget.
Image 4.1.3.7 signup_form_widget.dart SizedBox widget containing the ElevatedButton widget.
Image 4.1.3.8 signup_footer_widget.dart
Image 4.1.3.9 user_model.dart.
Image 4.1.3.10 import package_cloud_firestore_cloud_firestore.dart.
Image 4.1.3.11 ToJSON method.
Image 4.1.3.12 Factory Constructor Snapshot.
Image 4.1.3.13 final_data.
Image 4.1.3.14 User Repository_dart.
Image 4.1.3.15 final_db and static user_repository.
Image 4.1.3.16 createUser.
Image 4.1.3.17 ImagetoJSon conversion from usermodel and userrepository.
Image 4.1.3.18 getUserDetails.
Image 4.1.3.19 signup controller_dart.
Image 4.1.3.20 userRepo Get.Put.
Image 4.1.3.21 CreateUser controller.
Image 4.1.3.22 CreateuserModel with CreateuserController.
Image 4.1.3.25 Authentication_repository_dart.
Image 4.1.3.24 late final Rx.
Image 4.1.3.25 Void onReady.
Image 4.1.3.26 Ever Firebase User.
Image 4.1.3.27 loginfuture.
Image 4.2.2.1 locker system hold screen.
Image 4.2.2.2 locker system QR screen.
Image 4.2.2.3 locker system locker selection screen.
Image 4.2.3.1 Core Imports.
Image 4.2.3.3 The main run method.
Image 4.2.3.2 Core class and function.
Image 4.2.3.5 Lb_Operational Class.
Image 4.2.3.6 open_cam.
Image 4.2.3.7 checking for valid user.
Image 4.2.3.7 locker_select.
Image 4.2.3.8 pi_open_locker.
Image 4.2.3.9 Raspberry PI to servo connection.
Image 4.2.3.10 Locker box design for 3D printing.

1 Introduction

1.1 Scope Review Bryan France

The project addresses Australia's food waste problem by developing an eco-friendly and cost-effective food waste system for apartment residents. Currently, Australia wastes over 7 million tonnes of food annually, amounting to approximately $36 billion (Department of climate change, energy, the environment and water, 2022). Our solution involves residents creating an account using a mobile app and receiving a personalized QR code to access a locker bank for depositing their daily food waste. The system aims to promote recycling and reduce residents' environmental impact.

The user system was developed using the flutter framework with the dart language. It includes a user-friendly front end connected to a Google Firestore database. The database stores user data such as usernames, names, phone numbers, email addresses, and passwords. During account creation, a unique QR code is generated and associated with each user's profile. This QR code is displayed on the app's dashboard screen for easy access.

The locker bank system utilizes a Raspberry Pi running a Python script to perform a sequence of events. When a user scans their QR code, the system checks for an available locker, displays the assigned locker on the screen, and activates the lock mechanism, which is a servo motor connected to the Raspberry Pi. The script also sends relevant data, including the time of day, date, locker number, and user ID, to the database.

While the detailed database structure is outlined in the report, it is important to note that certain data stored, such as the user password, is for testing purposes only and will not be retained in its unencrypted form in the live version of the system. Ensuring data security and ethical practices are paramount.

By simplifying the user experience and minimizing user actions, the project encourages active participation. Users simply need to open the app, scan their QR code, swap containers, and contribute to the environment effortlessly.

2 System Requirements Review

2.1 Function Requirements Review

- Any registered user can access empty lockers
- Waste management company has access to lockers
- User account is required to use the system
- QR code will be provided for every user
- Locker will have scanner eye to scan the QR code
- Tenants able to access system from app
- Notification sent to collector when locker used
- Indicator device (possibly light) to show locker full / empty (Optional)
- New clean container left in locker after removing dirty used

container

- Full locker is only accessible by waste management company

We achieved nearly all of our function requirements. The ones we didn't manage to fulfill were on the edge of our scope, these were requirements like the notification system or the waste management company. We built our products with these items in mind though and where possible placeholder options were added.

2.2 Non-Functional Requirement Review

● Only registered users will have access [reliability requirement]

● User account requires code to activate [reliability requirement]

● We have full access to all user data [reliability requirement]

● Waste management company organizes cleaning and sanitizing of used containers [usability requirement]

● Tenants/Users should know how a phone app works [usability requirement]

● Tenants/Users should know what a QR code and how it works [usability requirement]

● Tenants/Users should know the system procedure (pamphlet /instructions on locker) [usability requirement]

● Tenants/User should know what constitutes acceptable food waste [usability requirement]

● We are only handling food waste [usability requirement]

● Waste pickup can be on set schedule or as available [performance requirement]

● Scales to check the carbon emission of tenants will be added eventually [performance requirement]

● Payment details for tenants will be added eventually (Such as PayPal/card details) [performance requirement]

● Hamburger / 3 dots for the User Interface (UI) menu will be added eventually [performance requirement]

● Maintenance of lockers will be a joint responsibility [supportability requirement]

● Security of lockers e.g.: camera, alarm may be required [supportability requirement]

The non-functional requirements are similar to the functional. All of the requirements were taken into consideration during development and either the features have been implemented or placeholder options are in place.

3 Database

3.1 Database Review Bryan France

The team initially planned to utilize an SQL relational database for the project. However, during our research on suitable tools, we came across Google's Firebase, specifically Google Firestore. Firestore is a NoSQL document-oriented database offered by Google Cloud Platform (GCP). It follows a document-based data model, where data is organized into collections, serving as containers for individual documents. Each document consists of key-value pairs, resembling JSON objects. This flexible schema allows for data storage, retrieval, and querying without imposing a rigid structure. Furthermore, Firebase offers user-friendliness and cost-effectiveness, which align with our goals of delivering a high-quality product. Notably, Firebase is open-source and free to use (to a certain point), providing added benefits to our project.

3.2 Database layout and Functionality Bryan France

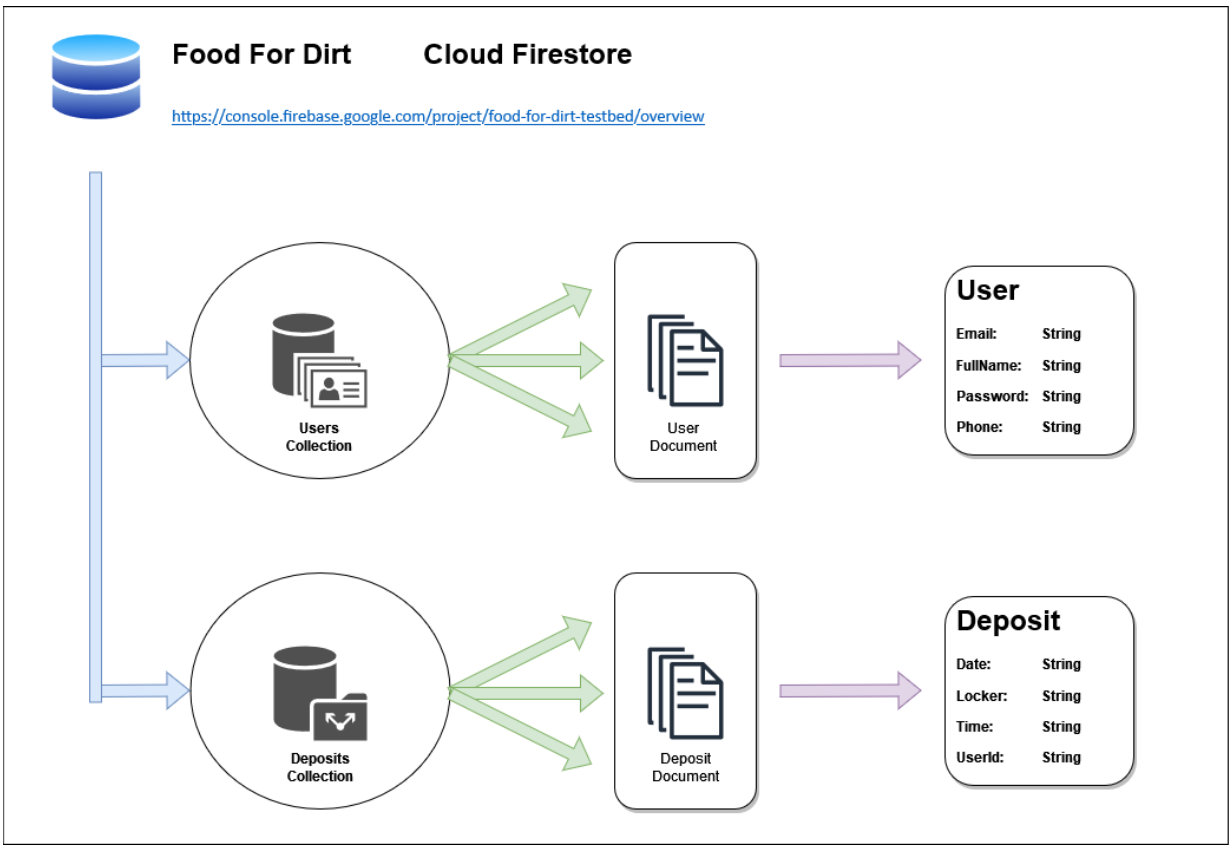


Image 3.2.1 Firestore document orientated database diagram.

The various parts of the firestore database are:

Firebase Authentication

- Email (required)
- Password (required)

This is a secure section of firestore only accessible by an administrator. The user email is used as a unique identifier, but each user is also given a unique user ID. The passwords stored here are encrypted and secure.

User Collection

- FullName
- Email
- Phone Number
- Password (will be removed for security)

Each user will have their own document created on signup via user application the information is taken from the text form fields on the signup screen (this is explained in greater detail in the app subsystem section). The app can be easily configured to take and record different data. The app currently stores the user password, this is only in place during testing and creation of the app for ease of use by the development team. It would be unethical to store this kind of data in an unencrypted state and will be removed before the app is put into a live format.

Deposits collection – each use of locker system creates a document containing.

- Date
- Time
- Locker Used
- User Id

The data collected from the locker system is basic information. In further development of the system our intention would be to gather as much information as possible. This would be used to generate a broader picture of the environmental impact this system can have. This data could then be turned into reports to show those positive outcomes.

4 System Architecture

4.1 User App System Peter Hans

User App Git - https://github.com/fwvu/UI-database

4.1.1 Technologies Used Peter Hans

The App our group created consists of a Splash screen to welcome users to our application, after the splash screen is displayed, user is taken to the home screen where user can either sign in if they don't have an account or log in if they already have an account

- **Android studio**: Android studio is an IDE (Integrated Development Environment) developed by Google. Android Studio is part of Android open-source project, so it is easily accessible for our group to build our android application using this program.
- **Microsoft Visual Studio Code:** Microsoft Visual Studio Code (VSC) is source code editor made by Microsoft, our group uses both android studio and VSC because there were some problems with android studio sometimes, these problems range from minor such a certain command won't get recognized in VSC (such as pub get) to major problems in which android studio unable to emulate phones, but VSC is able to emulate it. Although most of the app is still developed on android studio.
- **GitHub:** According to their about page website GitHub is an internet code sharing platform for software developers. According to their website GitHub is the complete developer platform to build, scale, and deliver secure software (GitHub, Inc., 2019). Our group decided to use GitHub to share our android studio codes because it is ease of use and our group have a small degree of familiarity with this website
- **Flutter:** According to their Homepage *Flutter is an open-source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase.* (Flutter, 2019). Our group chose to use flutter due to its ease of use and its open-source nature of the development kit.
- **Dart:** According to their overview page *Dart is a client-optimized language for developing fast apps on any platform. Its goal is to offer the most productive programming language for multi-platform development, paired with a flexible execution runtime platform for app frameworks* (Dart Overview, n.d.).
- **Firebase:** According to their Homepage *Firebase is an app development platform that helps you build and grow apps and games users love. Backed by Google and trusted by millions of businesses around the world (Google, 2023)*. Our group chose to use firebase due to its open-source nature and ease of use compared to the SQL relational database.

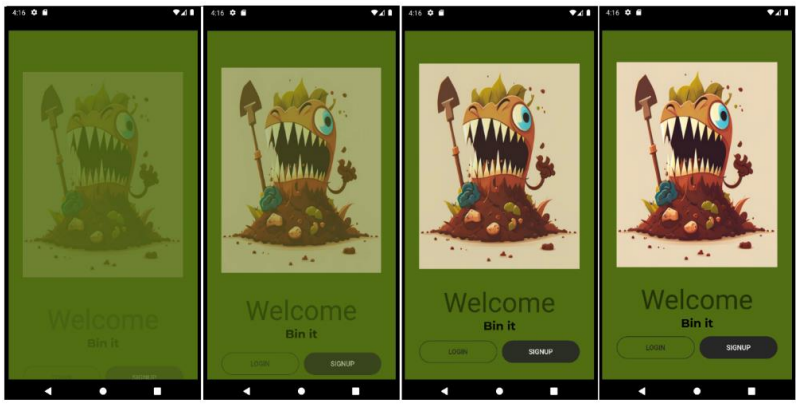
4.1.2 UI (User Interface) Descriptions Peter Hans

There are 7 screens in total for our user application. What follows is a brief description of each screen. They are the Splash screen, Welcome screen, Sign In screen, Sign Up screen, Dashboard, Profile and Edit Profile screens.

Animated Splash Screen

Image 4.1.2.1 Splash Screen animation fade in fade out.

The splash screen is the first screen loaded on the app. It fades in and out using a custom animation widget model we created. There are 2 images, a circle shape and text, all controlled from a start coordinate to an end coordinate with a time delay. After the splash screen executes it will then check for if a user is logged in to decide the next screen.



Animated Welcome Screen

Image 4.1.2.2 Welcome Screen animation.

Image 4.1.2.3 Welcome Screen.

Welcome Screen

Used when the app is started immediately after the splash screen. The two buttons underneath the picture will go to sign up and login when clicked.

The Login Screen

When login is clicked on the homepage, users will be able to enter their email address and password into their respective text boxes. When a user clicks on the sign up and forget password link. It will redirect users to their respective pages.

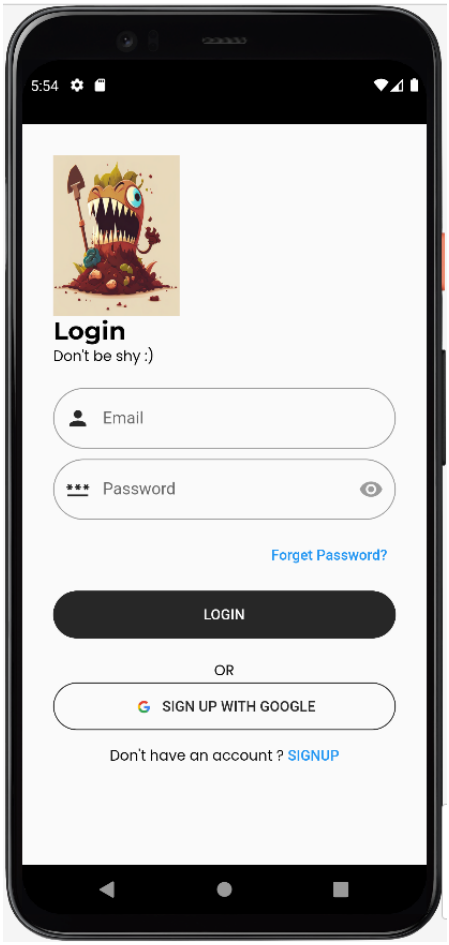


Image 4.1.2.4 Login Page Screen.

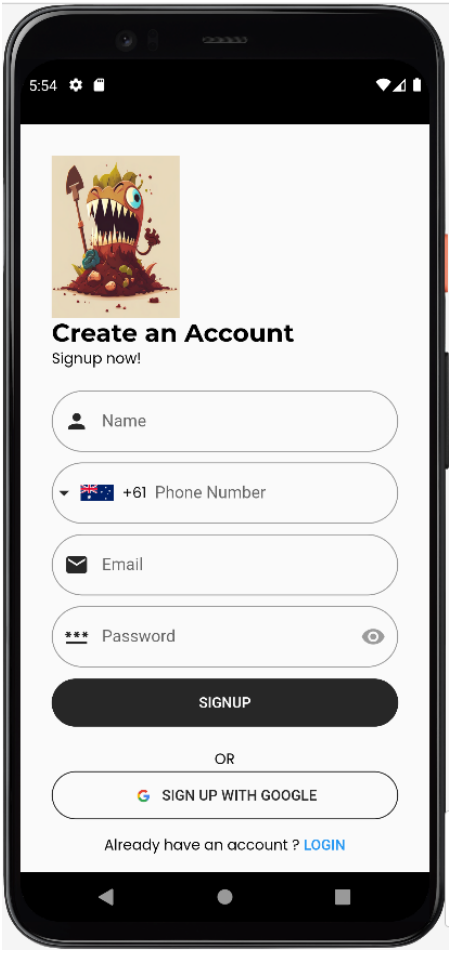


Image 4.1.2.5 Sign Up Screen.

The Sign-Up Screen

When Signup is clicked on the homepage, users will be able to enter their email address and password into their respective text boxes. When a user clicks on the sign up. It will redirect users to their respective pages.

The Dashboard Screen

The app bar has a drawer menu which is a placeholder and nonfunctional, the product name and the person icon which is used to access the edit profile screen. The main body consists of the welcome message and the unique personal QR code, also a user statistics button however this is nonfunctional.

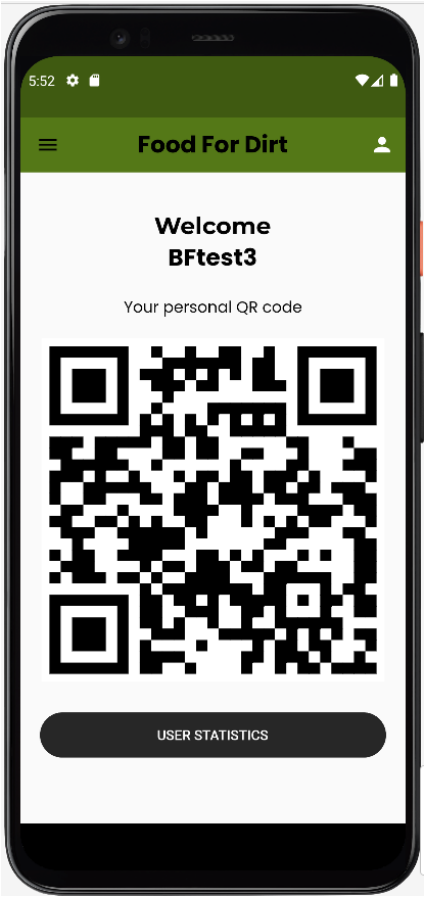


Image 4.1.2.6 Dashboard Screen.

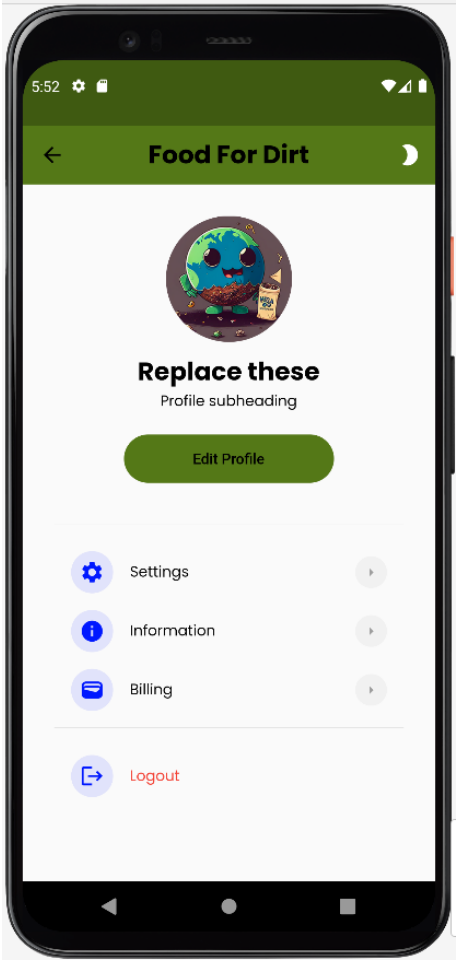


Image 4.1.2.7 Profile Screen.

The Profile Screen

When the logo person on the top right-hand corner of the screen is clicked. The profile screen includes placeholder logo to be populated with data later down the line. The edit profile takes user to the edit profile screen while logout button will logout the currently logged in user.

The Edit Profile Screen

When edit profile button is clicked on the profile screen, users can make changes to their name, email, phone number, and password. When update profile button is clicked, it will update user profile.

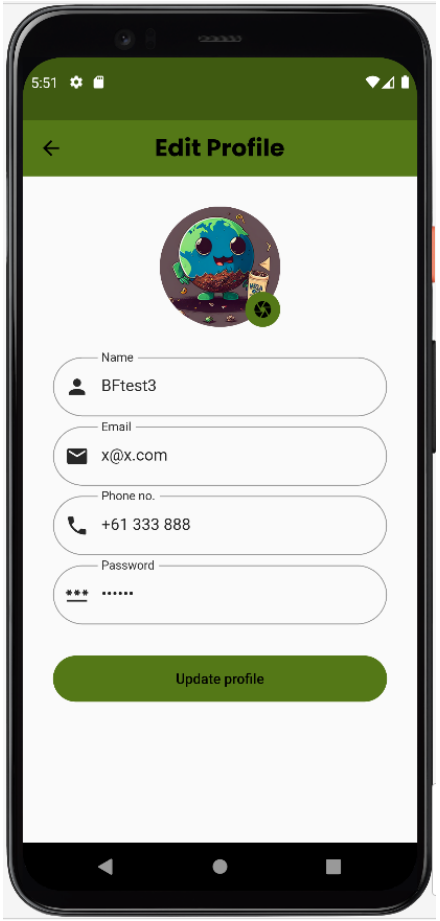


Image 4.1.2.8 Edit Profile Screen.

4.1.3 App Subsystems Explained Peter Hans, Bryan France

The user app consists of hundreds of files as the android studio template for creating a new flutter project builds out the base project for multiple options and uses. The files we created are mostly held in the /src folder. We built out the app trying to stick to a MVC (model view controller) style structure, this allowed us to keep the screen code separated from the logic. The team created several “constant” files too, we set up variables used throughout the app in these constant files so changing one thing would change it wherever it was used.

In this section we will break down the logic used in the main subsystems and some of the subsystems.

Screens:

All the screens are built in a comparable way. Flutter framework uses a scaffolding system of widgets nested in widgets nested in widgets. Rather than explain all the screens We will break down one screen. The signup screen and see how it is built, the widgets and custom widgets used.

The signup_screen.dart:

```
import 'package:flutter/material.dart';
import 'package:uidb/src/common_widgets/form/form_header_widget.dart';
import 'package:uidb/src/constants/image_strings.dart';
import 'package:uidb/src/constants/sizes.dart';
import 'package:uidb/src/constants/text_strings.dart';
import 'package:uidb/src/features/authentication/screens/signup/signup_widgets/signup_footer_widget.dart';
import 'package:uidb/src/features/authentication/screens/signup/signup_widgets/signup_form_widget.dart';
```

Image 4.1.3.1 signup_screen.dart imports.

First, all the imports are added to the file, these can be additional packages or other created files, but all are required for the signup screen to function.

```
class SignupScreen extends StatelessWidget {
  const SignupScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: Scaffold(
```

Image 4.1.3.2 signup_screen.dart class.

We create a class that returns scaffold. All screen widgets are nested in the scaffold. The class used is a stateless widget, we kept most of our screens stateless as this makes the app more efficient and responsive as stateless widgets do not require many resources or memory and can be thought of as static. Some of our screens are required to be stateful, which can be thought of as dynamic.

```
    return SafeArea(
      child: Scaffold(
        body: SingleChildScrollView(
          child: Container(
            padding: const EdgeInsets.all(bpdDefaultSize),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: const [
                FormHeaderWidget(image: bpdWelcomeImage, title: bpdSignupTitle, subTitle: bpdSignupSubTitle),
                SignupFormWidget(),
                SignupFooterWidget(),
              ],
            ),  // Column
          ),  // Container
        ),  // SingleChildScrollView
      ),  // Scaffold
    );  // SafeArea
  }
} // end of class
```

Image 4.1.3.3 signup_screen.dart body.

The scaffold is an outer widget which holds all the nested body widgets. We can see first is a **SingleChildScrollView** widget, this works as another outer shell and allows the screen to extend past the edge of the screen. Next, we nest a **Container** so we can set styling to the total screen. We then nest a C**olumn** widget to house the actual displayed widgets. There are 3 widgets on this screen, they are 3 custom widgets we created: a Header. a Form/body, and a Footer.

CUSTOM WIDGETS - THE HEADER.

```
1  import 'package:flutter/material.dart';
2
3  class FormHeaderWidget extends StatelessWidget {
4    const FormHeaderWidget({Key? key,
5      required this.image,
6      required this.title,
7      required this.subTitle,
8      this.textAlign,
9    }) : super(key: key);
10
11   final String image,title,subTitle;
12   final TextAlign ? textAlign;
13
14   @override
15   Widget build(BuildContext context) {
16     final size = MediaQuery.of(context).size;
17     return Column(crossAxisAlignment: CrossAxisAlignment.start,
18       children: [
19         Image(image: AssetImage(image), height: size.height * 0.2),
20         Text(title, style: Theme.of(context).textTheme.headlineMedium,),
21         Text(subTitle, textAlign: textAlign , style: Theme.of(context).textTheme.bodyMedium,),
22       ],
23     );  // Column
24   }
25 }
```

Image 4.1.3.4 form_header_widget.dart.

The header is its own class called in the signup_screen.dart. When called it requires 3 arguments an Image, a Title, and a Subtitle. it can be seen in *Image 4.1.3.3* on line 22. We pass in the arguments that point to the required images and text (held in the constants files) used for the screen.

CUSTOM WIDGETS - THE FORM/BODY.

```
9
10  class SignupFormWidget extends StatelessWidget {
11    const SignupFormWidget({super.key,});
12
13    @override
14    Widget build(BuildContext context) {
15
16      final auth = Get.put(AuthenticationRepository());
17      final controller = Get.put(SignUpController());
18      final formkey = GlobalKey<FormState>();
19
20      return Container(
21        padding: const EdgeInsets.symmetric(vertical: bpdFormHeight - 10),
22        child: Form(
23          key: formkey,
24          child: Column(
25            crossAxisAlignment: CrossAxisAlignment.start,
26            children: [
27              TextFormField(...),  // TextFormField
34              const SizedBox(height: bpdFormHeight - 20),
35              /*...*/
42              IntlPhoneField(...),  // IntlPhoneField
52              const SizedBox(height: bpdFormHeight - 20),
53              TextFormField(...),  // TextFormField
65              const SizedBox(height: bpdFormHeight - 20),
66              TextFormField(...),  // TextFormField
81              const SizedBox(height: bpdFormHeight - 20),
82              SizedBox(...),  // SizedBox
103           ],
104         ),  // Column
105       ),  // Form
106     );  // Container
107   }
108 }
```

Image 4.1.3.5 signup_form_widget.dart.

This is the overall signup form widget called by the signup screen. A stateless class which sets up some variables used in the form, returning a **Container** (scaffold). The **Container** has a **Form** widget which contains a **Column** widget which contains 4 **TextFormField** widgets and a **SizedBox** widget to house the **Button** widget.

```
children: [
  TextFormField(
    controller: controller.fullName,
    decoration: const InputDecoration(
      label: Text(bpdFullName),
      prefixIcon: Icon(Icons.person),
    ),  // InputDecoration
  ),  // TextFormField
```

Image 4.1.3.6 signup_form_widget.dart Text Form Field widget.

An example of the **TextFormField** widgets used in the signup form. It has a controller added to be able to use the data that is entered into the field. And the rest is styling.

```
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: () async {
      if (formkey.currentState!.validate()) {
        // email and password signup
        await auth.createUserWithEmailAndPassword(controller.email.text.trim(), controller.password.text.trim());

        // Get User info and pass it to controller to place in firestore
        final user = UserModel(
          email: controller.email.text.trim(),
          password: controller.password.text.trim(),
          fullName: controller.fullName.text.trim(),
          phoneNo: controller.phoneNo.text.trim(),
        );  // UserModel
        SignUpController.instance.createUser(user);
      }
    },
    child: Text(bpdSignup.toUpperCase()),
  ),  // ElevatedButton
),  // SizedBox
```

Image 4.1.3.7 signup_form_widget.dart SizedBox widget containing the ElevatedButton widget.

The **SizedBox** at the bottom of the form is used as a parent widget to the **ElevatedButton** widget. The **ElevatedButtom** widget has an **onPressed** function, the If condition checks for a valid **formstate** (all required text fields are used). If valid, it uses the email and password and create user function from the auth controller (explained in a database connection section) to create an authentication. It also creates a user from the **UserModel** model and creates a record of all data entered, stored in our firestore database.

CUSTOM WIDGETS - THE FOOTER

```dart
class SignupFooterWidget extends StatelessWidget {
  const SignupFooterWidget({super.key,});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const Text("OR"),
        SizedBox(
          width: double.infinity,
          child: OutlinedButton.icon(
            onPressed: (){},
            icon: const Image(image: AssetImage(bpdGoogleSignupImage), width: 20.0,),
            label: Text(bpdSignupWithGoogle.toUpperCase()),
          ),  // OutlinedButton.icon
        ),  // SizedBox
        TextButton(
          onPressed: (){
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const LoginScreen())
            );
          },
          child: Text.rich(...),  // Text.rich
        ),  // TextButton
      ],
    );  // Column
  }
}
```

Image 4.1.3.8 signup_footer_widget.dart.

The footer is another class but only contains a few buttons and some text. The first button is made of an icon image, the google logo this will allow signup via a google login. Second is a text that reads" already have an account" and a clickable **TextButton** "logon" when clicked this will reroute to the login screen.

The signup screen could have been created using only 1 file but would have resulted in so many nested widgets it would be hard to navigate. This is why we have broken up our screens into custom widgets for each section of the main screens. It keeps the code manageable, easily readable, and neat.

DATABASE CONNECTION AND INTERACTION MVC:

The Database consists of interconnected files to help communicate with the database. We use GetX, a state management package to handle routing and the CRUD (Create,Read,Update,Delete) functionality needed for this program to work.

This is a complex system, this is a general overview for connection of the database.

USER MODEL

```
class UserModel {
  final String? id;
  final String fullName;
  final String email;
  final String phoneNo;
  final String password;

const UserModel ({
  this.id,
  required this.email,
  required this.password,
  required this.fullName,
  required this.phoneNo,

});
```

Image 4.1.3.9 user_model.dart.

The code above represents a class of UserModel. This userModel maps the values of users and can be called to retrive or create database records.

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

Image 4.1.3.10 import package_cloud_firestore_cloud_firestore.dart.

It imports cloud firestore functionality to provides access to the cloud database.

```
toJson() {
  return {
    "FullName": fullName,
    "Email": email,
    "Phone": phoneNo,
    "Password": password,
  };
}
```

Image 4.1.3.11 ToJSON method.

This function inside the userModel is required in order for firebase to function because firebase will not accept a class as a functionality. For the program to work, it needs to be converted to JSON (Dictionary) format. This is because Firestore database store information in documents in a Key:Value pair system.

```
factory UserModel.fromSnapshot(DocumentSnapshot<Map<String, dynamic>> document) {
  final data = document.data()!;
  return UserModel(
    id: document.id,
    email: data["Email"],
    password: data["Password"],
    fullName: data["FullName"],
    phoneNo: data["Phone"],
  );
}
```

Image 4.1.3.12 Factory Constructor Snapshot.

This function is needed because firestore provides us with the data inside documentSnapshot. For this reason, it must be converted to our user model for it to work with our app. Factory object at the start is required because Dart allows us to create multiple constructors. But only when paired with factory function.

```
final data = document.data()!;
```

Image 4.1.3.13 final_data.

This line retrieves the data from a documentSnapshot object and assigns it to a data variable. It provides us with a complete object of the typical string value (Such as email,password,etc).

USER REPOSITORY

```dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:uidb/src/features/authentication/model/user_model.dart';

class UserRepository extends GetxController {
  static UserRepository get instance => Get.find();

  final _db = FirebaseFirestore.instance;

  createUser(UserModel user) async {
    await _db
        .collection("Users")
        .add(user.toJson()).whenComplete(() => Get.snackbar(
              "Success",
              "Your Account Has Been Created",
              snackPosition: SnackPosition.BOTTOM,
              backgroundColor: Colors.blue.withOpacity(0.1),
              colorText: Colors.cyan),
        )
        .catchError((error, stackTrace) {
      Get.snackbar("Error", "Something went wrong,try again",
          snackPosition: SnackPosition.BOTTOM,
          backgroundColor: Colors.redAccent.withOpacity(0.1),
          colorText: Colors.red);
      print(error.toString());
    });
  }


  Future<UserModel> getUserDetails(String email) async {
    final snapshot = await _db.collection("Users").where("Email", isEqualTo: email).get();
    final userData = snapshot.docs.map((e) => UserModel.fromSnapshot(e)).single;
    return userData;
  }
```

Image 4.1.3.14 User Repository_dart.

The UserRepository class provides methods to create a user and retrieve user details based on email using Cloud Firebase.

```dart
static UserRepository get instance => Get.find();

final _db = FirebaseFirestore.instance;
```

Image 4.1.3.15 final_db and static user_repository.

static UserRepository get instance => Get.find();: Defines a static get instance that allows the database to access the UserRepository instance using UserRepository.instance.

final _db = FirebaseFirestore.instance;: This creates an instance of FirebaseFirestore, which gives access to the Firestore database service.

```
createUser(UserModel user) async {
  await _db
      .collection("Users")
      .add(user.toJson()).whenComplete(() => Get.snackbar(
            "Success",
            "Your Account Has Been Created",
            snackPosition: SnackPosition.BOTTOM,
            backgroundColor: Colors.blue.withOpacity(0.1),
            colorText: Colors.cyan),
      )
      .catchError((error, stackTrace) {
    Get.snackbar("Error", "Something went wrong,try again",
        snackPosition: SnackPosition.BOTTOM,
        backgroundColor: Colors.redAccent.withOpacity(0.1),
        colorText: Colors.red);
    print(error.toString());
  });
}
```

Image 4.1.3.16 createUser.

The createUser method is responsible for creating a user in the Firestore database. After the Json conversion, If account creation is successful, it will display “Success, your Account has been Created” on the snackbar. Otherwise, it will pop up with the error message.

```
createUser(UserModel user) async {
  await _db
      .collection("Users")
      .add(user.toJson()).whenComplete(() => Get.snackbar(
            "Success",
            "Your Account Has Been Created",
            snackPosition: SnackPosition.BOTTOM,
            backgroundColor: Colors.blue.withOpacity(0.1),
            colorText: Colors.cyan),
      )
```

```
17
18  });
19
20  toJson() {
21    return {
22      "FullName": fullName,
23      "Email": email,
24      "Phone": phoneNo,
25      "Password": password,
26    };
27  }
```

Image 4.1.3.17 ImagetoJSon conversion from usermodel and userrepository.

It takes a UserModel object as a parameter representing the user to be created. Inside the method, it adds the user's data to the "Users" collection in Firestore using the add method. It converts the user object to JSON format using the toJson method from the UserModel class.

```
Future<UserModel> getUserDetails(String email) async {
  final snapshot = await _db.collection("Users").where("Email", isEqualTo: email).get();
  final userData = snapshot.docs.map((e) => UserModel.fromSnapshot(e)).single;
  return userData;
}
```

Image 4.1.3.18 getUserDetails.

The getUserDetails method retrieves user details from the Firestore database based on the provided email. It queries the "Users" collection using the where method to filter the documents where the "Email" field is equal to the provided email. It then retrieves the document snapshot using the get method.

The retrieved snapshot is mapped to a UserModel object using the fromSnapshot factory constructor from the UserModel class. Since the query is expected to return a single document based on user's individual emails, the single method is used to obtain a single UserModel object. Lastly, it returns the user data.

```
class SignUpController extends GetxController {
  static SignUpController get instance => Get.find();

  final email = TextEditingController();
  final password = TextEditingController();
  final fullName = TextEditingController();
  final phoneNo = TextEditingController();

  final userRepo = Get.put(UserRepository());

  Future<void> createUser(UserModel user) async {
    await userRepo.createUser(user);
    Get.to(() => const Dashboard());
  }
```

Image 4.1.3.19 signup controller_dart.

the SignUpController class handles user input for email, password, full name, and phone number fields, and provides a method to create a user by using user_repository.dart.From the above code, it can be broken down/explained as such:

- **static SignUpController get instance => Get.find();:** This line defines a static getter instance that allows accessing the SignUpController instance using SignUpController.instance.
- **Final(x) = Text editing Controller();** This line declares a TextEditingController called (x) that can be used to handle user input for the (x) field.

```
final userRepo = Get.put(UserRepository());
```

Image 4.1.3.20 userRepo Get.Put.

This line creates an instance of the UserRepository class using GetX dependency injection and assigns it to the userRepo variable. It allows accessing the UserRepository instance from other parts of the code.

```
Future<void> createUser(UserModel user) async {
  await userRepo.createUser(user);
  Get.to(() => const Dashboard());
}
```

Image 4.1.3.21 CreateUser controller.

This method is responsible for creating a specific user. It takes a UserModel object as a parameter, representing which specific user to be created.

```
createUser(UserModel user) async {
  await _db
      .collection("Users")
      .add(user.toJson()).whenComplete(() => Get.snackbar(
            "Success",
            "Your Account Has Been Created",
            snackPosition: SnackPosition.BOTTOM,
            backgroundColor: Colors.blue.withOpacity(0.1),
            colorText: Colors.cyan),
      )
```

```
final email = TextEditingController();
final password = TextEditingController();
final fullName = TextEditingController();
final phoneNo = TextEditingController();

final userRepo = Get.put(UserRepository());

Future<void> createUser(UserModel user) asyn
  await userRepo.createUser(user);
  Get.to(() => const Dashboard());
```

Image 4.1.3.22 CreateuserModel with CreateuserController.

Inside the method, it calls the createUser method of the userRepo instance to find the specific user in the database. Finally, it navigates to the Dashboard page using Get.to method for routing.

AUTHENTICATION REPOSITORY

```dart
class AuthenticationRepository extends GetxController {
  static AuthenticationRepository get instance => Get.find();

  //variables
  final _auth = FirebaseAuth.instance;
  late final Rx<User?> firebaseUser;

  @override
  void onReady() {
    Future.delayed(const Duration(seconds: 6));
    firebaseUser = Rx<User?>(_auth.currentUser);
    firebaseUser.bindStream(_auth.userChanges());
    ever(firebaseUser, _setInitialScreen);
  }


  _setInitialScreen(User? user) {
    if (user == null) {
      Get.offAll(() => const SplashScreen());
    } else {
      Get.offAll(() => Dashboard(uid:user.uid));
    }
  }


  Future<void> createUserWithEmailAndPassword(
      String email, String password) async {
    try {
      await _auth.createUserWithEmailAndPassword(
          email: email, password: password);
      firebaseUser.value != null
          ? Get.offAll(() => const Dashboard())
          : Get.to(() => const WelcomeScreen());
    } on FirebaseAuthException catch (e) {
      final ex = SignUpWithEmailAndPasswordFailure.code(e.code);
      Get.snackbar("FIREBASE AUTH EXCEPTION -", (ex.message),
        backgroundColor: Colors.redAccent,
        snackPosition: SnackPosition.BOTTOM,
        titleText: const Text("Account creation failed",
        style: TextStyle(color: bpdWhiteColor),),  // Text
        messageText: Text(ex.message,
```

Image 4.1.3.23 Authentication_repository_dart.

The AuthenticationRepository class provides methods for creating a user, logging in a user, and logging out a user using Firebase Authentication. It also handles the initial screen navigation based on the user's initial state, whether they're logged in or not.

```
final _auth = FirebaseAuth.instance;
late final Rx<User?> firebaseUser;
```

Image 4.1.3.24 late final Rx.

final _auth = FirebaseAuth.instance;: This line creates an instance of FirebaseAuth using the instance property, which gives access to the Firebase Authentication service.

late final Rx<User?> firebaseUser;: This line declares a late-initialized Rx<User?> variable called firebaseUser. Rx is a GetX variable that replaces the string function (It comes with auth_repo package). While late final is required because the command is initialized in onReady instead of inside the class function. This command functionally allows the user to remain logged in even when the app is closed (This command relates to the onReady command).

```
@override
void onReady() {
  Future.delayed(const Duration(seconds: 6));
  firebaseUser = Rx<User?>(_auth.currentUser);
  firebaseUser.bindStream(_auth.userChanges());
  ever(firebaseUser, _setInitialScreen);
}
```

Image 4.1.3.25 Void onReady.

The onReady method is an override from the GetX library, which will be called whenever authentication repository is ready. The code can be broken down as such:

- **Future.delayed(const Duration(seconds: 6));**: It adds a delay of 6 seconds before executing the code. This delay is used for splash screen.
- **firebaseUser = Rx<User?>(_auth.currentUser);**: It checks whether user is logged in into firebase when accessing the app. If user is logged on, it immediately grants access to the account and vice versa.
- **firebaseUser.bindStream(_auth.userChanges());**: It registers the activity of users. In conclusion, this command tracks what user is doing, whether they're going to click on logout or signout. It will inform the firebaseUser to perform certain actions. It basically informs firebase whether the user is signed out or logged out and keeps the user logged in even when the device/app is closed/turned off. Only when the user signed/logged out then it will logout the users in question.

```
  ever(firebaseUser, _setInitialScreen);
}


_setInitialScreen(User? user) {
  if (user == null) {
    Get.offAll(() => const SplashScreen());
  } else {
    Get.offAll(() => Dashboard(uid:user.uid));
  }
}
```

Image 4.1.3.26 Ever Firebase User.

ever(firebaseUser, _setInitialScreen);: **ever** is an action command whenever anything new/updated is registered within firebase. It tracks whether user is logged in or signed out. If user is signed out, it will take them to the initial screen which is the splash screen then the homepage. While if the user is signed/logged in, it will take them to the dashboard with the user specific user id.

```
Future<void> loginUserWithEmailAndPassword(
    String email, String password) async {
  try {
    await _auth.signInWithEmailAndPassword(email: email, password: password);
  } on FirebaseAuthException catch (e) {
  } catch (_) {}
}
```

Image 4.1.3.27 loginfuture.

The loginUserWithEmailAndPassword method is responsible for authenticating a user with an email and password using Firebase Authentication. If a FirebaseAuthException occurs, it registers the exception. Then it will display an error message provided by firebase.

If any other exception occurs, it catches the exception and displays an error message provided by firebase.

4.2 Locker Bank System Bryan France

Locker Bank Git - https://github.com/fwvu/LockerBank

4.2.1 Technologies Used Bryan France

The locker system we've created is a simple program consisting of a splash/hold screen, a trigger to start scanning the QR code, once it has scanned a valid code it then cycles through a list of lockers until it finds the next available, then it displays on screen the assigned locker and will be connected to a servo motor to open the locker.

I wrote the locker system program using VScode as the IDE. I chose the python language as this system would require electronic components and a raspberry pi uses python well to control said components.

Python is a good programming language for the simple operation the locker system will perform, but it also requires a few extra modules to perform all the tasks required. The libraries added were:

- **DATETIME** - a standard module used for accurate timestamps.
- **OpenCV2** - OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in commercial products. Being an Apache 2 licensed product, OpenCV makes it easy for businesses to utilize and modify the code. (OpenCV, 2020). Opencv2.
- **PILLOW** - User-friendly fork of PIL (Python Imaging Library) developed by Jeffrey A. Clark and contributors, expands the image processing capabilities of Python. With extensive support for various file formats (Pillow, n.d.). Used to resize and reformat images and video in the program.
- **TKINTER** - A standard GUI (Graphical User Interface) toolkit. It provides a set of modules and classes that enable developers to create desktop applications with interactive graphical interfaces. Tkinter is based on the Tcl/Tk framework, (Tkinter, n.d.) which allows for the integration of widgets and other visual elements into our program. With Tkinter, we can design and manipulate windows, buttons, menus, and various other components. The main component used to generate display screens for the user to receive prompts on what to do with the locker system.
- **Pyzbar** – contains functions used to read the QR code.
- **FIREBASE** – The firebase library allows us to connect to firestore database created and used by the user app. The locker system is connected to firestore to record data about the usage of the locker system.

4.2.2 UI (User Interface) Descriptions Bryan France

The hold screen is the initial screen is used when the locker system program is started. The buttons at the bottom will be removed in the production level program but are in place now for testing and ease of use. The “Full Test” button will trigger the automatic operation of the program, we need to have another meeting with Thinh to discuss if he would like a touch screen “Start” button or a movement sensor.

Image 4.2.2.1 locker system hold screen.

The second screen in the sequence, the video display created by openCV will try to detect and decode a QR code. It uses a bounding box to lock onto the 3 large squares (anchor points) on the top left and right and bottom left of the QR image. It currently displays the decoded code on screen too, this will be removed before putting into production.

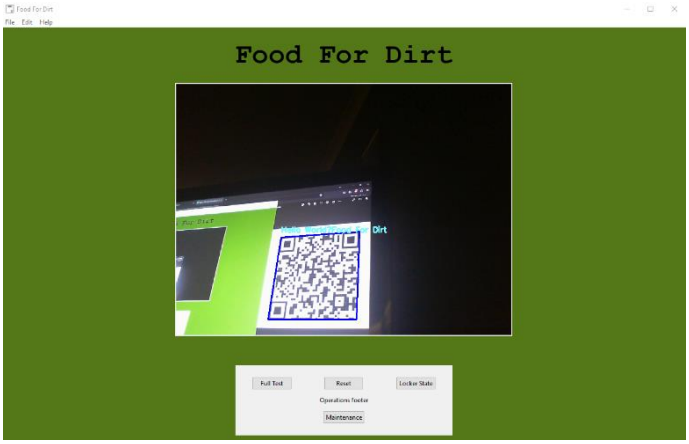
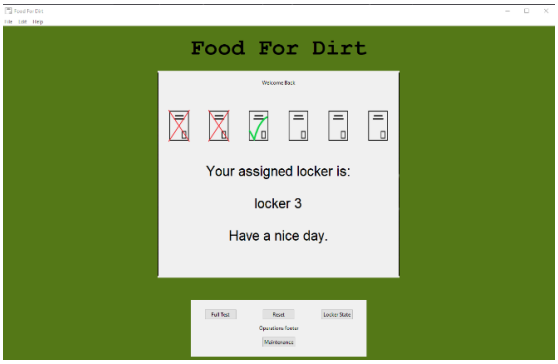


Image 4.2.2.2 locker system QR screen.



The third screen is displayed on the locker system. Once a valid QR code is read on screen 2 it then performs a check for the next available locker and outputs the display of the chosen locker. This screen will be updated with better images and text before being put into production.

Image 4.2.2.3 locker system locker selection screen.

4.2.3 Locker Bank Subsystems Explained Bryan France

The locker System consists of several files. I have tried to comment on as much of the code as possible. The report that follows will show all the components of the program but only highlight certain notable sections and their purpose. It starts with a driver file LB_Core.py.

lb_Core.py:

```python
import tkinter as tk
from tkinter import ttk

from lb_maintenance import *
from lb_operational import *
from lb_assets import *
from lb_menu import *
```

Image 4.2.3.1 Core Imports.

```python


# RUN
if __name__ == "__main__":
    app = Core()
    app.title(appTitle)
    app.geometry(f'{mainWinSizeX}x{mainWinSizeY}')
    app.iconbitmap(iconImg)
    app.config(bg=appGreen)
    app.mainloop()
```

Image 4.2.3.3 The main run method.

```python
11  # Open a standard window
12  class Core(tk.Tk):
13      def __init__(self, *args, **kwargs):
14          tk.Tk.__init__(self, *args, **kwargs)
15
16          menuBar = menu_bar(self)
17          self.config(menu=menuBar)
18
19          # HEADER
20          self.appName = tk.Label(self, text=appTitle, font="courier 40 bold", background=appGreen)
21          self.appName.pack(pady=20)
22
23          # creating a frame and assigning it to container
24          container = tk.Frame(self, height=400, width=600)
25          # specifying the region where the frame is packed in root
26          container.pack(side="top", fill="both", expand=True)
27
28          # configuring the location of the container using grid
29          container.grid_rowconfigure(0, weight=1)
30          container.grid_columnconfigure(0, weight=1)
31
32          # dictionary of frames
33          self.frames = {}
34
35          # list to store the 2 screens
36          for F in (lbOperational, lbMaintenance):
37              cont = F.__name__
38              frame = F(container, self)
39              self.frames[cont] = frame
40              frame.grid(row=0, column=0, sticky="nsew")
41
42          # first displayed frame
43          self.show_frame('lbOperational')
44
45      def show_frame(self, cont):
46          frame = self.frames[cont]
47          # raises the current frame to the top
48          frame.tkraise()
```

Image 4.2.3.2 Core class and function.

The Core class creates the parent window using tkinter. It adds a menu_bar widget (tkinter) to the exterior of the window. Then adds a header element (label widget tkinter) taking the top portion of the window. A container (frame widget) is added to the rest of the window. This container can be used to create as many screens as required. 2 screens are added to a dictionary of frames LbOperational and LbMaintanence.

Using the **show_frame** function allows the tkraise method to place a screen at the front of the frame. It is called on line 43 to raise the screen LbOperational setting it as the first displayed screen.

Lb_Operational.py:

```
9   # Operational Frame -- main way public interacts with system
10  class lbOperational(tk.Frame):
11      def __init__(self, parent, controller):
12          tk.Frame.__init__(self, parent, bg=appGreen)
13          self.controller = controller
14
15          # set up camera object called cam which we will be used with OpenCV
16          self.cam = None
17
18          # BODY FRAME
19          self.bodyScreen = tk.Label(self, text="Body Frame", image='')
20          self.bodyScreen.pack()
21
22          self.hold_img()
23
24          # FOOTER FOR TESTING -- REMOVE or add to screen menu--
25
26          self.footer = tk.Label(self, text="Operations footer")
27          self.footer.pack(side="bottom", pady=10)
28
29          startButton = ttk.Button(self.footer, text="Full Test",  command=lambda: open_cam(self))
30          startButton.grid(row=1, column=0, pady=20, padx=30)
31
32          resetButton = ttk.Button(self.footer, text="Reset",  command=lambda: close_cam(self))
33          resetButton.grid(row=1, column=2, pady=20, padx=30)
34
35          checkButton = ttk.Button(self.footer, text="Locker State",  command=lambda: locker_select(self))
36          checkButton.grid(row=1, column=3, pady=20, padx=30)
37
38
39          switch_window_button = ttk.Button(self.footer, text="Maintenance", command=lambda: controller.show_frame('lbMaintenance'))
40          switch_window_button.grid(row=2, column=2, pady=20, padx=30)
41
42          ##################################
```

Image 4.2.3.5 Lb_Operational Class.

The LbOperational class is a frame widget child of the Core class. Inside this frame is created the bodyScreen (label widget) this is used to display the changing screens, the hold_img function is called to fill the frame with an initial display. A footer (label widget) is added to house buttons used for testing; this will be removed in production. Line 16 sets up the empty camera object so video can be passed to the frame.

3 functions are added as children of the LbOperational class.

clear_bodyScreen is used to empty the bodyScreen frame.

hold_img first calls clear_bodyScreen. Then it resizes a png file using PILLOW and ImageTK.PhotoImage so that tkinter can process the image. The image is added to the bodyScreen frame.

update_video_img. This function is created because tkinter does not understand video. So, each image from the camera is taken, converted, and resized using PILLOW and imageTK.PhotoImage. The image is then added to the bodyScreen.

Lb_Op_Commands.py:

This acts as a kind of controller file. But is more a sequence of events.

The open_cam is the first function called when the start button is pressed. This will start the openCV video capture and call update_video.

```
def open_cam(self):
    delete_img(self)
    self.cam = cv2.VideoCapture(0)
    update_video(self)
```

Image 4.2.3.6 open_cam.

update_video checks for an instance of the video. It first sets up some variable self.detector will be used to detect and decode the QR code, and both ret and cam_image will use the cam.read function of openCV. If the video is running the variables data, bbox and ret are setup to capture the decoded information.

The code uses numpy to plot the coordinates of the bounding box to track and target the QR code. If no data is found the update_video_img is passed the single frame of the video cam_image to be displayed in the bodyScreen then update_video calls itself a kind of loop is created, and it will continue to update images until data is found.

When data is found, it will decode the box's contents and add it to the data variable. The processedData variable splits the contents of data and checks for “Food_For_Dirt” we use this as part of our QR code to validate the code is for our system. The second part of the split data will contain a users ID and is set to a global variable userId to be used later. Once a valid QR code is read and the data is processed the locker_select function is called.

```
if data:
    # splits the qr code data
    processedData = data.split()
    # check the first part of the QR code contains food_for_dirt
    if processedData[0] == qrId:
        # creates a global userID so it can be passed when locker is selected
        global userId
        userId = processedData[1]
        # triggers locker select function
        locker_select(self)
    # if the QR decodes Maintenance it will perform a locker reset
    elif processedData[0] == maintenanceId:
        locker_reset(self)
```

Image 4.2.3.7 checking for valid user.

Locker_select opens the lockertest.dat file and stores the contents in a dictionary. It then checks through the dictionary to find the first value the is set to “empty” and changes it to “avail” then saves out to the same lockertest.dat file. And calls the locker_state function.

```python
def locker_select(self):
    lockers = pickle.load(open('lockertest.dat', 'rb'))
    found_empty = False
    chosenLocker = None

    for key, value in lockers.items():
        if value == "empty":
            lockers[key] = "avail"
            found_empty = True
            break

    if found_empty:
        pickle.dump(lockers, open('lockertest.dat', 'wb'))
        locker_state(self)
        if chosenLocker:
            pi_open_locker(chosenLocker)
    else:
        lockers_full(self)
```

Image 4.2.3.7 locker_select.

Locker_state opens the lockertest.dat file and stores the contents in a dictionary. It creates some tkinter frames for text on screen. The function loops through the key values pairs of the dictionary and adds an image to tkinter frames based on the value “empty” “avail” and “full” after that it will find values that are set to “avail” adds the key to chosenLocker and set them to “full”. It then passes userId and chosenLocker to lb_firebase.firebase_deposit and triggers the pi_control function.

Pi_control will trigger the servo to rotate which acts as a locking mechanism for the physical locker.

```python
def pi_open_locker(locker):
    _lockerNum = locker.split()
    lockerNum = _lockerNum[1]
            # call      # GPIO      # start freq          # stop freq
    servo = AngularServo(lockerNum, min_pulse_width=minpw, max_pulse_width=maxpw)

    servo.angle = 90
    sleep(2)
    servo.angle = 0
```

Image 4.2.3.8 pi_open_locker.

Firebase_deposit takes the userId and the locker number and creates a database record of the usage with user ID, locker number, time, and date.

We will add a reed switch to the locker door that will track when the locker is open and when it is closed again. After closing the system will restart.

Hardware:

The locker system is built to run on a small computer or as we have done for testing a Raspberry PI 3 b+. This hardware would most likely be outsourced to a fabricator but using the PI allow us to test functionality. We attached a small servo motor to act as a locking mechanism.

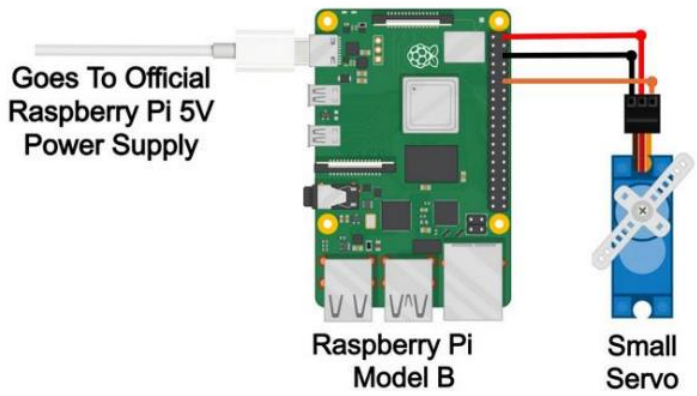


Image 4.2.3.9 Raspberry PI to servo connection.

We were also lucky enough to have a 3D printed locker created, to be used as a prototype and for our demonstration. It's a simple design done in tinkerCAD a very easy to use online design tool.

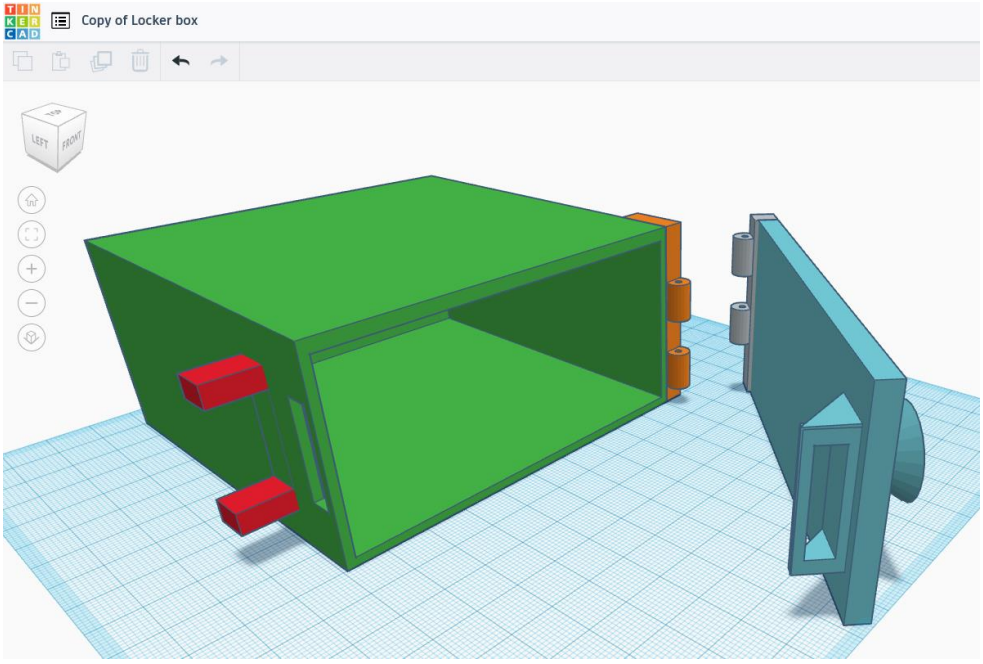


Image 4.2.3.10 Locker box design for 3D printing.

5 Challenges and Problems Peter Hans, Bryan France

Development of the mobile app:

- Unfamiliarity with Android studio and firebase causes some confusion when doing the project. At first, our group were having difficulties at the start such as android studio is incapable of installing certain plugins such as flutter, java and Device manager for the phone emulator itself. Our group figured that we needed to restart android studio for it to recognize the installation of certain plugins and the device manager. Not only that, but our group also didn't have the basic understanding of firebase at the time. Only after a few months and thoroughly researching about firebase thoroughly (such as rule conditions, controllers, models and repositories) do our group understand the relationship between firebase and android studio.

- OTP development failed to work as intended. Our group at first was planning to develop a simple OTP message to notify individuals that may have forgotten their password/wishes to reset their password. However, due to complications and our coordinator's suggestion not to include OTP. Our group abandoned this idea.

- Developing the connection from firebase to Android Studio with firebase rulesets. There were some issues with the relationship between firebase and android studio when our group were developing the app to connect to firebase, one of these issues is what rule can be applied to our app. At first our group turns off security which uses rule version 0. Then we tried rule version 1, but it doesn't apply the changes we made in the app to firebase. Only after we tried rule 2 which is the latest version and making the changes to read/write permissions and changing the rule permissions to null (instead of specifying the if conditions to a certain degree) does the android studio and firebase have a successful connection.

Reflection/changes made for the betterment of the app:

Our group changed the generic error messages generated by firebase and google to our own custom error messages.

Development of the Locker System:

- Tkinter was a huge learning curve when dealing with individual screens/frames and getting them to interact with different functions. Getting outputs to the terminal was much easier. The biggest issue I faced was dealing with images and the video. Tkinter is an original component of python and uses an old file format. Each time I wanted to use a more common image format, I would have to pass the image into imageTK with PIL and then resize and output the image bac to tkinter. This was also the case with video tkinter does not understand video and would take each frame of the video passed into imageTK resized and formatted through PIL and passed bac to tkinter.

- Passing the images came with issues too because of the way I switched to a class model to try and organize the code into a MVC style architecture. I was trying to pass the camera image into the main bodyScreen but it would not display, I was sure I had written the code to be functional and tried many tweaks. I used Microsoft's Bing AI pasted in some of my code and asked it why my video was not showing. Annoyingly it spat out the answer straight away and changed 1 line of the code. Then proceeded to taunt me more by telling me in my code the Images were getting garbage collected before being sent to the class.

- I wrote the locker code to be usable without an internet connection as I was allowing for a connection failure and did not want it to lose track of the used lockers, I achieved this by saving the locker state as a dictionary locally to a file with Pickle (python library). Eventually this also allowed me to move away from a hardcoded number of lockers as it now takes the number directly from the saved file and creates a display from the stored dictionary.

Design:

- None of us are artistic and our attempts to produce images to be used in the apps hit a wall quickly. Since it is the new thing out there, we decided to try and task AI to generate some images for us. I signed up to Mid Journey, an AI that allows 20 free image prompts a month. The results were quite good, and all the images used through the UI app and locker system are all AI generated.

6 Student Contributions

Student ID	Name	Contribution (#)	%
s3892805	Bryan France	Team Leader	55

As the team lead, I took on various responsibilities throughout the project to ensure its successful completion. I focused on effective communication, project management, software selection, and development progress tracking. Additionally, I actively contributed to the development of both the mobile app and the locker system.

Communication and Project Management:

I created new communication channels on Discord and archived the project plan channels. I developed a detailed Gantt chart to outline the project timeline and milestones. This allowed us to track our progress effectively.

To keep everyone updated on the project's status, I created a progress report document in Office365. The document was organized into sections, covering different subsystems and features. By regularly updating this document, we maintained a clear overview of the project's progress.

Software Choices and Development:

After careful research, we decided to use the Flutter framework with Dart language in the Android Studio IDE for mobile app development. To ensure consistency, I assisted the team in setting up their development environments correctly, ensuring smooth workflows throughout the development process.

To gain familiarity with Flutter, both Peter and I built basic apps for practice and learning purposes. I started developing the main project app by creating the splash screen, welcome screen and established a well-defined folder structure for the project and added theme files, text styles, and placeholder images.

I figured out and set up a GitHub repository, enabling the team to work on the app at the same time and have version control.

Mobile App Progress:

During the development phase, we worked together to understand how Firebase worked and integrated it into the app. I left Peter working on the authentication sections of the app and focused on developing the dashboard and incorporating QR code functionality.

To ensure app distribution and testing, I created an APK of the app and installed it on one of my old phones. This allowed us to evaluate its functionality and user experience.

Locker System Development:

I moved on to development of the locker system. Initially, I created a functional version that provided outputs in the terminal. Once the basics were established, I implemented a graphical user interface (GUI) using Tkinter. I refined and utilized the same functions developed earlier, I then acquired a Raspberry Pi from Assefa and set it up with the required electronic components. Additionally, I created a GitHub repository for the locker system.

Documentation and Reporting:

I created a formal progress report document to be submitted. I edited and formatted the rough progress reports, incorporating meeting diaries, database schema, design roughs, and other relevant details. The document was prepared and shared ready for submission.

I created the final report document and added to; the introduction, scope review, database review and schema, and sections related to both the mobile app and locker system. Challenges and problems encountered during the development process were documented as well.

I setup a basic layout for a user manual document, and set peter to work adding content. I added in details for raspberry pi installation.

Ongoing Project Management:

Throughout the project, I continued to update the Gantt chart, ensuring that it accurately reflected our progress and milestones. This allowed us to monitor our project timeline and make necessary adjustments.

<table>
<tr><td colspan="4">Overall, as the team lead, I ensured effective communication, project management, and coordinated development efforts. My contributions to the mobile app development, establishment of the locker system, documentation, and ongoing project management played a crucial role in the project's success.</td></tr>
<tr><td>s4643570</td><td>Peter Hans</td><td>Team member</td><td>45</td></tr>
<tr><td colspan="4">As a project member, I ensured the setting up and completion of the database development (Firebase), authentication (Model, Views, Controllers) and the development of the user manual, poster and video required for the project.

Firebase:

I am responsible for setting up the firebase database (in terms of account creation) and creating the models, views and controllers alongside the commands necessary to ensure connection between firebase and android studio.

Android studio:

I am also responsible on creating the authentication commands (Such as authentication controllers, user repositories and user model). I am also responsible in creating the login with the functional go to sign in page button (Created that button for the sign in as well) and created the attempted (but failed since it’s non-functional requirement of our project) OTP and email for forgot password. (which is functional but turned off at the moment).

Documentation:

My contribution to the final report documentation are the requirements (Both functional and non-functional), technologies used for the mobile app development, the MVC connection and interaction between the databases (such as controllers and views) and the required references for the technologies used.

User Manual:

I am also responsible with the editing and completion of the user manual with the troubleshooting that needs to be stamped out/bypassed for future references.</td></tr>
</table>

Poster: I am also responsible with the design and the contents of the poster required for this project, also done the fact searching required for the poster's call to action. Video: I am also responsible with designing and editing our demonstration video required for this project. Overall, as team member, I ensured proactive measures to complete this project and followed my team leaders' suggestions and input for this project to succeed.			
s4652882	Darcy Williams	Team Member	0
Removed self from project (personal)			

A complete project portfolio can be found at:

https://livevuedu-my.sharepoint.com/:f:/g/personal/bryan_france_live_vu_edu_au/EgxbPCypmyxBlrPEHW0U-OwBNhy5-lCkN3kQIVup9Bapbw?e=JkaB77

7 References

Department of climate change, energy, the environment and water. (n.d.). https://www.dcceew.gov.au/environment/protection/waste/food-waste

Cloud app System:

Google. (2023). Firebase. Firebase. https://firebase.google.com/

GitHub, Inc. (2019). Build software better, together. GitHub. https://github.com/about

Flutter. (2019). Flutter - Beautiful native apps in record time. Flutter.dev. https://flutter.dev/

Dart (n.d.). Dart.dev. https://dart.dev/overview

Android Developers. (2018). Android Developers. https://developer.android.com/

Microsoft. (2019). Visual Studio. Visual Studio. https://visualstudio.microsoft.com/

Locker System:

OpenCV. (2020, November 4). About - OpenCV. https://opencv.org/about/

Pillow. (n.d.). Pillow (PIL Fork). https://pillow.readthedocs.io/en/stable/

Tkinter. (n.d.). Python Documentation. https://docs.python.org/3/library/tkinter.html

Opencv lite build instructions (n.d.). https://qengineering.eu/install-opencv-lite-on-raspberry-pi.html

Rust installer (m.d). https://www.rust-lang.org/tools/install

Ai Images:

Midjourney. (n.d.). Midjourney. https://www.midjourney.com/

Online tools:

Microsoft 365. (n.d.). https://www.office.com/

Discord | your place to talk and hang out. (n.d.). Discord. https://discord.com/

Tinkercad | From mind to design in minutes. (n.d.). Tinkercad. https://www.tinkercad.com/

Canva. (n.d.). *Free design tool: presentations, video, social media* https://www.canva.com/en_au/

Quick and easy video editor | Clipchamp. (2023, May 2). Clipchamp. https://clipchamp.com/en/